Třídící algoritmy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

05.03.2026

CC BY-NC-SA 4.0

Úvod

Třídění

Třídění (sorting) je jedním z nejzákladnějších problémů v informatice

Proč je důležité?

- Binární vyhledávání vyžaduje **seřazené pole**
- Databáze: ORDER BY, indexy
- Odstranění duplicit
- Mnoho algoritmů vyžaduje seřazená data jako vstup

Probírané algoritmy

Algoritmus	Průměrná složitost	Typ
Bubble Sort	$\mathcal{O}(n^2)$	Jednoduchý
Selection Sort	$\mathcal{O}(n^2)$	Jednoduchý
Insertion Sort	$\mathcal{O}(n^2)$	Jednoduchý
Quick Sort	$\mathcal{O}(n \log n)$	Pokročilý
Merge Sort	$\mathcal{O}(n \log n)$	Pokročilý

Klíčové pojmy

Stabilní třídění

- Prvky se **stejnou hodnotou** si zachovají **vzájemné pořadí** jako ve vstupním poli
- Příklad: řadíme studenty podle známky – stabilní algoritmus zachová abecední pořadí studentů se stejnou známkou

In-place třídění

- Algoritmus nepotřebuje **žádnou extra paměť** (kromě konstantního počtu proměnných)
- Paměťová složitost $\mathcal{O}(1)$

Klíčové pojmy

Comparison-based (porovnávací) třídění

- Řadí prvky pouze pomocí **porovnání** dvou prvků ($<, >, =$)
- Dolní mez složitosti porovnávacího třídění je $\Omega(n \log n)$
  - Žádný porovnávací algoritmus nemůže být asymptoticky lepší

Adaptive (adaptivní) algoritmus

- Využívá existující uspořádání ve vstupních datech
- Na předtříděném poli je rychlejší

Bubble Sort

Princip

Bubble Sort třídí pole opakovaným **porovnáváním sousedních prvků** a jejich prohazováním

Největší prvek „probublá“ na konec pole

Postup:

1. Projdi pole zleva doprava
2. Porovnej každé dva sousední prvky
3. Pokud jsou ve špatném pořadí, prohoď je
4. Opakuj, dokud není pole seřazené

Příklad – Bubble Sort

Pole: `[5, 3, 8, 1, 2]`

1. průchod:

Start	5	3	8	1	2
Porovnej 5,3 → swap	3	5	8	1	2
Porovnej 5,8 → ok	3	5	8	1	2
Porovnej 8,1 → swap	3	5	1	8	2
Porovnej 8,2 → swap	3	5	1	2	8

Příklad – Bubble Sort

2. průchod: `[3, 5, 1, 2, | 8]`

Start	3	5	1	2	8
Porovnej 3,5 → ok	3	5	1	2	8
Porovnej 5,1 → swap	3	1	5	2	8
Porovnej 5,2 → swap	3	1	2	5	8

Po 2. průchodu: `[3, 1, 2, | 5, 8]`

Příklad – Bubble Sort

3. průchod: `[3, 1, 2, | 5, 8]`

Start	3	1	2	5	8
Porovnej 3,1 → swap	1	3	2	5	8
Porovnej 3,2 → swap	1	2	3	5	8

Po 3. průchodu: `[1, 2, | 3, 5, 8]`

Příklad – Bubble Sort

4. průchod: `[1, 2, | 3, 5, 8]`

Start	1	2	3	5	8
Porovnej 1,2 → ok	1	2	3	5	8

Žádný swap → pole je seřazené!

1	2	3	5	8

Optimalizace – noswap

Problém: základní Bubble Sort vždy provede $n - 1$ průchodů, i když je pole seřazené dříve

Řešení: na začátku každého průchodu nastavíme příznak `swapped = false`

- Pokud během průchodu dojde k **alespoň jednomu swapu** → `swapped = true`
- Pokud **žádný swap** neproběhl → pole je seřazené, **ukončíme algoritmus**

Optimalizace – noswap

Efekt:

- Na již seřazeném poli stačí **1 průchod** $\rightarrow \mathcal{O}(n)$
- Bez optimalizace by i seřazené pole trvalo $\mathcal{O}(n^2)$

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – pole seřazené pozpátku
- Průměrný případ: $\mathcal{O}(n^2)$
- Nejlepší případ: $\mathcal{O}(n)$ – pole je již seřazené (díky `swapped` optimalizaci)

Vlastnosti

- **Stabilní** – neprohazuje prvky se stejnou hodnotou
- **In-place** – $\mathcal{O}(1)$ extra paměti
- **Adaptivní** – díky optimalizaci s `swapped`
- V praxi se používá pouze pro **výukové účely**

Selection Sort

Princip

Selection Sort opakovaně **vybírá minimum** z neseřazené části a umísťuje ho na správnou pozici

Postup:

1. Najdi **nejmenší** prvek v neseřazené části pole
2. Prohoď ho s prvním prvkem neseřazené části
3. Posuň hranici seřazené části o 1 doprava
4. Opakuj

Příklad – Selection Sort

Pole: `[5, 3, 8, 1, 2]`

Start	5	3	8	1	2	
$i = 0, \min = 1$	5	3	8	1	2	swap(0,3)
$i = 1, \min = 2$	1	3	8	5	2	swap(1,4)
$i = 2, \min = 3$	1	2	8	5	3	swap(2,4)
$i = 3, \min = 5$	1	2	3	5	8	ok

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$
- Průměrný případ: $\mathcal{O}(n^2)$
- Nejlepší případ: $\mathcal{O}(n^2)$ – **vždy stejné!**

Na rozdíl od bubble sortu, selection sort **nemá optimalizaci pro předseřazená pole**

Vlastnosti

- **Nestabilní** – prohazování může změnit pořadí stejných prvků
- **In-place** – $\mathcal{O}(1)$ extra paměti
- **Maximální počet swapů** – maximálně $n - 1$ prohození

Insertion Sort

Princip

Insertion Sort buduje seřazenou část pole postupným **vkládáním prvků** na správnou pozici

Analogie: Řazení karet v ruce – každou novou kartu vložíte na správné místo mezi ostatní

Princip

Postup:

1. Začni od 2. prvku (1. prvek je „seřazený“)
2. Vezmi aktuální prvek (klíč)
3. Posuň všechny větší prvky v seřazené části o 1 doprava
4. Vlož klíč na uvolněné místo

Příklad – Insertion Sort

Pole: `[5, 3, 8, 1, 2]`

Start	5	3	8	1	2	
i=1: klíč=3	3	5	8	1	2	$3 < 5 \rightarrow$ vlož před 5
i=2: klíč=8	3	5	8	1	2	$8 > 5 \rightarrow$ zůstává
i=3: klíč=1	1	3	5	8	2	$1 < 3 \rightarrow$ vlož na začátek
i=4: klíč=2	1	2	3	5	8	$2 < 3 \rightarrow$ vlož za 1

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – pole seřazené pozpátku
- Průměrný případ: $\mathcal{O}(n^2)$
- Nejlepší případ: $\mathcal{O}(n)$ – pole je již seřazené

Vlastnosti

- **Stabilní** – zachovává pořadí stejných prvků
- **In-place** – $\mathcal{O}(1)$ extra paměti
- **Adaptivní** – rychlý na předtříděných datech
- **Online** – lze přidávat prvky za běhu
- V praxi **efektivní pro malá pole** (do 50 prvků)

Quick Sort

Princip

Quick Sort používá strategii **rozděl a panuj** (divide and conquer)

Postup:

1. Zvol **pivot** (referenční prvek)
2. **Rozděl** (partition) pole na dvě části:
   - Prvky **menší** než pivot → levá část
   - Prvky **větší** než pivot → pravá část
3. **Rekurzivně** seřaď obě části

Příklad – Quick Sort

Pole: `[5, 3, 8, 1, 2]`, pivot = poslední prvek

1. úroveň: pivot = **2**

< pivot	pivot	> pivot
`[1]`	**2**	`[5, 3, 8]`

`[1]` je seřazené, řadíme `[5, 3, 8]`

Příklad – Quick Sort

2. úroveň: `[5, 3, 8]`, pivot = **8**

< pivot	pivot	> pivot
`[5, 3]`	**8**	`[]`

3. úroveň: `[5, 3]`, pivot = **3**

< pivot	pivot	> pivot
`[]`	**3**	`[5]`

Volba pivotu

Volba pivotu **zásadně ovlivňuje výkon** quick sortu

1. První/poslední prvek

- Jednoduché, ale **nevýhodné na předseřazených polích** $\rightarrow \mathcal{O}(n^2)$

2. Náhodný prvek

- Nejčastěji používaná metoda
- Nejhorší případ je stále $\mathcal{O}(n^2)$, ale je **velmi nepravděpodobný**

Volba pivotu

3. Medián tří

- Vyber první, prostřední a poslední prvek
- Jako pivot zvol jejich **medián**
- Lepší ochrana proti nejhoršímu případu

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n^2)$ – špatná volba pivotu (vždy min/max)
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – pivot vždy dělí pole na poloviny

Vlastnosti

- **Nestabilní** – prohazování přes pivot mění pořadí
- **In-place** – $\mathcal{O}(\log n)$ paměti na zásobníku (rekurze)
- V praxi **nejrychlejší** pro velká pole
- Funkce `qsort()` ve standardní knihovně C (`<stdlib.h>`)

Merge Sort

Princip

Merge Sort také používá strategii **rozděl a panuj**, ale jiným způsobem

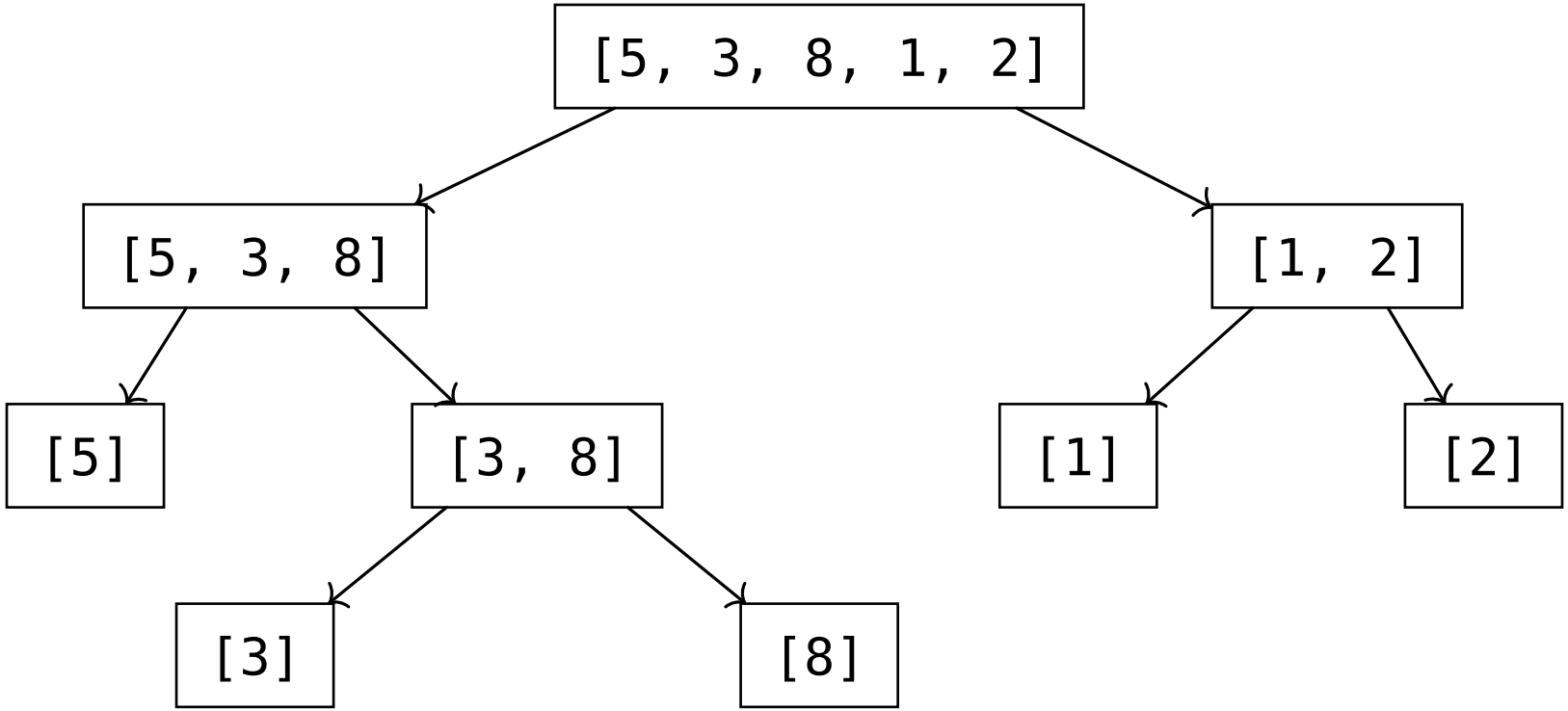
Postup:

1. **Rozděl** pole na dvě poloviny
2. **Rekurzivně** seřaď obě poloviny
3. **Slij** (merge) obě seřazené poloviny do jednoho seřazeného pole

Příklad – rozdělení

Pole: `[5, 3, 8, 1, 2]`

Příklad – rozdělení



Příklad – slévání

Seřazené části se **slévají** zpět dohromady:

Levá	Pravá	Výsledek
`[3]`	`[8]`	`[3, 8]`
`[5]`	`[3, 8]`	`[3, 5, 8]`
`[1]`	`[2]`	`[1, 2]`
`[3, 5, 8]`	`[1, 2]`	`[1, 2, 3, 5, 8]`

Vlastnosti

Složitost:

- Nejhorší případ: $\mathcal{O}(n \log n)$
- Průměrný případ: $\mathcal{O}(n \log n)$
- Nejlepší případ: $\mathcal{O}(n \log n)$ – **vždy stejné!**

Vlastnosti

- **Stabilní** – slévání zachovává pořadí stejných prvků
- **Není in-place** – potřebuje $\mathcal{O}(n)$ extra paměti
- **Garantovaná** složitost – žádný nejhorší případ jako u quick sortu
- Vhodný pro **spojové seznamy** (příští přednáška) - nepotřebuje random access

Srovnání

Přehledová tabulka

Algoritmus	Best	Average	Worst	Paměť	Stabilní	In-place
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Ano	Ano
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Ne	Ano
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Ano	Ano
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	Ne	Ano
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Ano	Ne

Kdy který algoritmus?

Malé pole (do 50 prvků)

- **Insertion sort** – jednoduchý, nízká režie, adaptivní

Velké pole, obecný případ

- **Quick sort** – v praxi nejrychlejší díky cache-friendly přístupu

Potřebuji garanci $\mathcal{O}(n \log n)$

- **Merge sort** a jiné – žádný nejhorší případ

Předtříděná data

- **Insertion sort** – $\mathcal{O}(n)$ v best case

Dolní mez třídění

Věta: Jakýkoliv **porovnávací** třídící algoritmus potřebuje v nejhorším případě alespoň $\Omega(n \log n)$ porovnání

Důkaz (intuice):

- Pro n prvků existuje $n!$ možných permutací
- Každé porovnání rozdělí možnosti na dvě skupiny
- Potřebujeme alespoň $\log_2(n!)$ porovnání
- $\log_2(n!) \approx n \log_2 n$ (Stirlingova aproximace)

Proto Quick Sort a Merge Sort jsou **asymptoticky optimální**